

Systèmes d'exploitation I

Module: M3 - INF-S3 & IAP-S3

Prof. R. HANNANE

r.hannane@uca.ma

Faculté des Sciences Senglalia, Marrakech
Université Cadi Ayyad



Les commandes de manipulation du contenu de fichier (Suite)

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

- `awk` : traite les fichiers de données textuels et les flux
 - ▶ Les **données d'entrée sont divisées en enregistrements et champs** que l'on peut traiter à l'aide de variables
 - ▶ Lorsqu'un **enregistrement est lu, il est placé dans la variable `$0`** (à ne pas confondre avec la variable `$0` de Bash et Perl)
 - ▶ L'**enregistrement est ensuite scindé en plusieurs parties dans un tableau interne**, allant de **`$1`** à **`$NF`**, en utilisant la variable **`FS`** comme séparateur de champs

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

1 Lecture avec awk :

- Pour lire les enregistrements d'un fichier :

```

abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ cat fl.txt
Les Filiere dispensees sont:
-----
Mathematique      SMA      120
Informatique      INF_IAP, SMI3, SMI6      250
Science_Physique  SMP      313
Informatique      SMI4, SMI5      110
Science_Chimie    SMC      280
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print $0}' fl.txt
Les Filiere dispensees sont:
-----
Mathematique      SMA      120
Informatique      INF_IAP, SMI3, SMI6      250
Science_Physique  SMP      313
Informatique      SMI4, SMI5      110
Science_Chimie    SMC      280
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ █

```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

2 Les variables `awk` :

- ▶ Un **nouveau séparateur d'enregistrement** peut être défini à l'aide de la variable **RS**
- ▶ Les **enregistrements** consistent en des champs séparés par le **séparateur de champs**
- ▶ Par défaut, les champs sont séparés par un espace blanc
- ▶ Les **champs de chaque enregistrement** sont référencés par le **caractère Dollar (\$)** suivi d'un **numéro de champ**, en commençant par 1
Le premier champ est représenté avec \$1, le second avec \$2, etc.
- ▶ Le **dernier champ** est référencé avec la **variable spéciale NF**
- ▶ L'**ensemble de l'enregistrement** peut être référencé avec **\$0**

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

- ▶ **Les principales variables de `awk` sont:**
 - **RS** (record separator): Le séparateur d'enregistrement par défaut est le caractère nouvelle ligne "`\n`"
 - **NR** (number of record): numéro d'enregistrement de la sortie de la commande `awk`
 - **FS** ou **OFS** (field separator): définit le séparateur de champs des sorties. Par défaut le séparateur est le caractère espace
 - **ORS** (output record separator): Séparateur d'enregistrement de la sortie
 - **NF** (number of fields): Si le délimiteur standard est un "espace blanc" pour les champs, cela correspondra au nombre de mots dans l'enregistrement en cours
 - **FILENAME**: Nom du fichier utilisé dans la commande
 - **\$0**: Représente la ligne entière d'un texte
 - **\$1**: Représente le première champs ...

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

3 Extraire des colonnes avec `awk` :

- ▶ Pour extraire les colonnes d'un fichier, on utilise les variables des champs
- ▶ Exemple: Extraire la 1ere colonne du fichier

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print $1}' fl.txt
Les
-----
Mathematique
Informatique
Science_Physique
Informatique
Science_Chimie
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

► Exemple: Extraire les colonnes 2 et 3 :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print $2 $3}' fl.txt
Filteredispensees

SMA120
INF_IAP,SMI3,
SMP313
SMI4,SMI5
SMC280
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print $2 " " $3}' fl.txt
Filtere dispensees

SMA 120
INF_IAP, SMI3,
SMP 313
SMI4, SMI5
SMC 280
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

► Pour spécifier le séparateur, on peut aussi utiliser l'option -F:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F"\t" '{print $2}' fl.txt

SMA
INF_IAP, SMI3, SMI6
SMP
SMI4, SMI5
SMC
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```


Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

4 Ajouter avec awk :

- ▶ Exemple: Pour ajouter un | entre chaque champs de sortie:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print $1 "|" $2 "|" $3}' FS="\t" fl.txt
Les Filiere dispensees sont:|
-----||
Mathematique|SMA|120
Informatique|INF_IAP, SMI3, SMI6|250
Science_Physique|SMP|313
Informatique|SMI4, SMI5|110
Science_Chimie|SMC|280
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ Pour ajouter une phrase dans la sortie avec un retour à la ligne :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print "Le 1er champ: ", $1, "\n Le 2eme champ: ", $2}' FS="\t" fl.txt
Le 1er champ: Les Filiere dispensees sont:
Le 2eme champ: -----
Le 1er champ: Mathematique
Le 2eme champ: SMA
Le 1er champ: Informatique
Le 2eme champ: INF_IAP, SMI3, SMI6
Le 1er champ: Science_Physique
Le 2eme champ: SMP
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

5 Recherche de texte et extraction de données avec **awk** :

- ▶ Afficher que des colonnes dont la valeur est égale à une valeur spécifique:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F"\t" '{if($1=="
Informatique") print NR $0}' fl.txt
4Informatique    INF_IAP, SMI3, SMI6      250
6Informatique    SMI4, SMI5      110
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ Pour rechercher un motif :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '/250/ {print $0}' fl.txt
Informatique    INF_IAP, SMI3, SMI6      250
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '/Informatique/ {print $0}'
fl.txt
Informatique    INF_IAP, SMI3, SMI6      250
Informatique    SMI4, SMI5      110
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ Pour rechercher dans un champs en particulier :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F"\t" '$3 ~ /250/ {print $0}'
fl.txt
Informatique    INF_IAP, SMI3, SMI6      250
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

6 Effectuer une comparaison numérique avec awk :

- ▶ Pour afficher que des colonnes dont la valeur est inférieure à une valeur spécifique :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F"\t" '$3<250 {print NR $0}'
fl.txt
1Les Filiere dispensees sont:
2-----
3Mathematique   SMA      120
6Informatique   SMI4, SMI5    110
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ Pour afficher le contenu d'un fichier dont les données de la 2eme colonne commencent par la lettre I :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F"\t" '$2 ~ /^I/ {print $0}'
fl.txt
Informatique   INF_IAP, SMI3, SMI6    250
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ Le caractère \$ est utilisé pour tester la fin d'une donnée:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F"\t" '$3 ~ /3$/ {print $0}'
fl.txt
Science_Physique   SMP      313
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec **awk**

7 Substitution avec **awk** :

- ▶ **Exemple:** Pour remplacer le séparateur ":" par la virgule ","

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print $1,$2}'
FS="\t" OFS=":" fl.txt
Les Filiere dispensees sont::
-----:
Mathematique:SMA
Informatique:INF_IAP, SMI3, SMI6
Science_Physique:SMP
Informatique:SMI4, SMI5
Science_Chimie:SMC
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ La Substitution est aussi possible grâce à la fonction **gsub** qui utilise une syntaxe similaire à **sed**
- ▶ Pour **remplacer le mot SMI par INF_IAP** :

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{gsub(/SMI/, "INF_IAP");
print $0}' fl.txt
Les Filiere dispensees sont:
-----
Mathematique      SMA      120
Informatique      INF_IAP, INF_IAP3, INF_IAP6      250
Science_Physique      SMP      313
Informatique      INF_IAP4, INF_IAP5      110
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

8 Les règles **BEGIN** et **END** de `awk` :

- ▶ La règle **BEGIN** est **exécutée une fois avant que tout traitement de texte ne commence**. Il est exécuté avant que `awk` ne lit même aucun texte
- ▶ La règle **END** est **exécutée après la fin du traitement**
- ▶ Les règles **BEGIN** et **END** seront exécutés dans l'ordre

⇒ Cela est utile pour afficher un élément en début ou fin, par exemple lorsque l'on souhaite effectuer des opérations mathématiques pour afficher le résultat

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

- On peut utiliser **BEGIN** pour afficher un texte en tout début de sortie:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SEI/TP3$ awk 'BEGIN{print "\n***Session Automne***" } FS="\t" {print $1, $3}' fl.txt
***Session Automne***
Les dispensees
-----
Mathematique 120
Informatique 250
Science_Physique 313
Informatique 110
Science_Chimie 280
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SEI/TP3$
```

- Pour afficher le nombre de lignes d'un fichier:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SEI/TP3$ awk 'END{print "\n Le fichier ",FILENAME,"
contient ",NR," lignes."} {print NR $0}' fl.txt
1Les Fillere dispensees sont:
2-----
3Mathematique SMA 120
4Informatique INF_IAP, SMI3, SMI6 250
5Science_Physique SMP 313
6Informatique SMI4, SMI5 110
7Science_Chimie SMC 280
Le fichier fl.txt contient 7 lignes.
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SEI/TP3$
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

9 Extraire des lignes d'un fichier avec `awk` :

► Extraire à partir de la 2eme ligne:

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk 'NR>2' fl.txt
Mathematique      SMA      120
Informatique      INF_IAP, SMI3, SMI6      250
Science_Physique  SMP      313
Informatique      SMI4, SMI5      110
Science_Chimie    SMC      280
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

► Extraire un intervalle de lignes: de la 1ere ligne à la 4eme ligne d'un fichier :

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk 'NR>=1 && NR<3' fl.txt
Les Filiere dispensees sont:
-----
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec `awk`

10 Convertir du texte en majuscules avec `awk` :

- ▶ La fonction `toupper` passe la sortie en majuscules
- ▶ Pour passer tout un fichier en majuscules :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{print
toupper($0)}' fl.txt
LES FILIERE DISPENSEES SONT:
-----
MATHEMATIQUE      SMA      120
INFORMATIQUE      INF_IAP, SMI3, SMI6      250
SCIENCE_PHYSIQUE  SMP      313
INFORMATIQUE      SMI4, SMI5      110
SCIENCE_CHIMIE    SMC      280

abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

- ▶ Pour passer la 1ere colonne en majuscules :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk -F="\t"
'{print toupper($1) " " $2}' fl.txt
LES FILIERE DISPENSEES SONT:
-----
MATHEMATIQUE      SMA      120
INFORMATIQUE      INF_IAP, SMI3, SMI6      250
SCIENCE_PHYSIQUE  SMP      313
INFORMATIQUE      SMI4, SMI5      110
```


Les commandes de manipulation du contenu de fichier

14) Contenu du fichier: Avec awk

11 Additionner les valeurs d'une colonne avec awk :

- ▶ SUM permet de faire des sommes de valeurs et notamment de colonnes
- ▶ Pour additionner la valeur de la 3eme colonne :

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$ awk '{SUM=SUM+$3}
{print $0} END{print "Total des etudiants: " SUM}' FS="\t" fl.txt
Les Filiere dispensees sont:
-----
Mathematique      SMA      120
Informatique      INF_IAP, SMI3, SMI6      250
Science_Physique  SMP      313
Informatique      SMI4, SMI5      110
Science_Chimie    SMC      280

Total des etudiants: 1073
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/TP3$
```

L'enchaînement des commandes

L'enchaînement des commandes

- **L'enchaînement des commandes** permet d'exécuter des commandes multiples à la suite d'une commande
- Il existe **trois types d'enchaînement**:
 - a) Le point virgule " ; "
 - b) Le symbole `&&`
 - c) Le symbole `||`

L'enchaînement des commandes

a) Le point virgule " ; "

● Chaîner les commandes

- ▶ Permet d'exécuter plusieurs commandes, sur une seule ligne, l'une à la suite de l'autre, et d'une façon indépendante
- ▶ Le résultat d'une commande ne va pas influencer sur l'exécution de l'autre

⇒ Pour se faire, il suffit de les séparer avec un point-virgule " ; "

- ▶ **Exemple1:** `date;pwd;cal`

L'enchaînement des commandes

a) Le point virgule " ; "

► Exemple2:

```

abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$ cal 1 2021; cal 10 2021; cal 11 2021
    January 2021
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
    October 2021
Su Mo Tu We Th Fr Sa
                1  2
 3  4  5  6  7  8  9
10 11 12 13 14 15 16
17 18 19 20 21 22 23
24 25 26 27 28 29 30
31
    November 2021
Su Mo Tu We Th Fr Sa
    1  2  3  4  5  6
 7  8  9 10 11 12 13
14 15 16 17 18 19 20
21 22 23 24 25 26 27
28 29 30

abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~$

```

L'enchaînement des commandes

b) Le symbole " && "

- Le symbole " && " agit comme un " AND " logique
- Si la première commande a réussi alors la deuxième sera exécutée si non il ne sera pas exécutée
- Exemple:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1$ ls /home/abdessamad/Desktop/SE1/test
fich1.txt test02.txt test1
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1$ ls /home/abdessamad/Desktop/SE1/test/test1
fich3.txt
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1$ ls /home/abdessamad/Desktop/SE1/test/test1
&& ls /home/abdessamad/Desktop/SE1/test
fich3.txt
fich1.txt test02.txt test1
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1$
```

L'enchaînement des commandes

c) Le symbole " || "

- Le symbole " || " agit comme un " OU " logique
- Si la première commande échoue la deuxième sera exécutée
- Si la première commande réussit la deuxième ne sera pas exécutée
- Exemple:

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1$ ls /home/abdessamad/Desktop/SE1/test/test1
|| ls /home/abdessamad/Desktop/SE1/test
fich3.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1$ ls /home/abdessamad/Desktop/SE1/test/test1
|| echo bonjour
fich3.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1$ ls /home/abdessamad/Desktop/SE1/test/test
|| echo bonjour
ls: cannot access '/home/abdessamad/Desktop/SE1/test/test': No such file or directory
bonjour
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1$
```

Redirections et les expressions régulières

Introduction

- Shell Linux offre plusieurs outils permettant de manipuler les fichiers textes (affichage , édition, recherche...)

- De plus, **Shell possède des moyens permettant de contrôler la sortie d'une commande**

⇒ **Au lieu d'afficher cette sortie sur la fenêtre du terminal on peut la rediriger vers un fichier ou vers une autre commande**

- **Comment?**

- ▶ Utilisation des pipes |
- ▶ Utilisation des redirections des commandes
- ▶ Manipuler les fichiers textes
- ▶ Utilisation des expressions régulières

Le pipe

- Généralement, **lorsqu'une commande génère un résultat ou une erreur, le résultat (ou l'erreur) est affiché à l'écran**
 - Le caractère **pipe (|)** est utilisé **pour envoyer la sortie (résultat) d'une commande à une autre** au lieu de l'afficher à l'écran
- ⇒ **La sortie d'une commande devient une entrée pour une autre commande**
- Le **pipe |** est surtout utilisé pour raffiner et filtrer le **résultat d'une commande initial**

Le pipe

Pour illustrer le mécanisme du piping dans les exemples qui vont suivre on va utiliser les commandes `head` et `tail`.

Remarque:

- Les commandes `head` et `tail` ne s'appliquent pas uniquement aux fichiers mais on peut les appliquées aussi sur la sortie d'une commande

Attention:

- Il est important de bien choisir l'ordre des commandes lorsqu'on utilise le piping

Le pipe

- **Exemple1:** Lister uniquement les 10 premiers fichiers contenus dans `/etc` et non la totalité des fichiers
⇒ Pour ce faire, on doit passer à la commande `head` la sortie de la commande `ls /etc`

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls /etc | head
acpi
adduser.conf
alsa
alternatives
anacrontab
anthy
apache2
apg.conf
apm
apparmor
```

⇒ Le résultat de `ls /etc` n'est pas affiché à l'écran mais sert d'entrée à la commande `head` qui va faire son traitement puis afficher le résultat à l'écran

Le pipe

- **Exemple2:** Numéroter les lignes de la sortie de `ls -l`
 → (la commande `nl` permet de numéroter les lignes de la sortie d'une commande)

```

abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls -l
total 40
-rw-rw-r-- 1 abdessamad abdessamad    0 Oct 30 11:19 f1.txt
-rw-rw-r-- 1 abdessamad abdessamad   24 Oct 22 16:07 fich1.txt
-rw-rw-r-- 1 abdessamad abdessamad   71 Oct 22 19:13 personne.txt
-rw-rw-r-- 1 abdessamad abdessamad  101 Oct 22 19:17 test02.txt
drwxrwxr-x 2 abdessamad abdessamad 4096 Oct 22 18:00 test1
-rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 19:58 TEST.tar
-rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 20:00 z
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls -l | nl
 1 total 40
 2 -rw-rw-r-- 1 abdessamad abdessamad    0 Oct 30 11:19 f1.txt
 3 -rw-rw-r-- 1 abdessamad abdessamad   24 Oct 22 16:07 fich1.txt
 4 -rw-rw-r-- 1 abdessamad abdessamad   71 Oct 22 19:13 personne.txt
 5 -rw-rw-r-- 1 abdessamad abdessamad  101 Oct 22 19:17 test02.txt
 6 drwxrwxr-x 2 abdessamad abdessamad 4096 Oct 22 18:00 test1
 7 -rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 19:58 TEST.tar
 8 -rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 20:00 z
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$

```

Le pipe

- **Attention: L'ordre des commandes lorsqu'on utilise le piping est très important**

- ▶ **Exemple1:** Afficher les 5 dernières lignes qui vont être numérotées de la commande `ls -l`

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls -l | nl | tail -5
 4 -rw-rw-r-- 1 abdessamad abdessamad   71 Oct 22 19:13 personne.txt
 5 -rw-rw-r-- 1 abdessamad abdessamad  101 Oct 22 19:17 test02.txt
 6 drwxrwxr-x 2 abdessamad abdessamad 4096 Oct 22 18:00 test1
 7 -rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 19:58 TEST.tar
 8 -rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 20:00 z
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

⇒ dans cet exemple la commande `ls -l` est exécutée puis son résultat est passé à la commande `nl` qui va numéroté les lignes du résultat de la commande précédente puis on a passé le résultat de `nl` à `tail -5` qui va afficher sur l'écran les 5 dernières lignes du résultat de `nl`

Le pipe

- ▶ **Exemple2:** le même exemple précédent mais on a inversé l'ordre des commandes `nl` et `tail -5`

Ici le résultat de `ls` est d'abord envoyé à `tail -5` qui va prendre 5 lignes de ce résultat puis il va les passer à `nl` qui va les numéroter de 1 à 5

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls -l | tail -5 | nl
1 -rw-rw-r-- 1 abdessamad abdessamad 71 Oct 22 19:13 personne.txt
2 -rw-rw-r-- 1 abdessamad abdessamad 101 Oct 22 19:17 test02.txt
3 drwxrwxr-x 2 abdessamad abdessamad 4096 Oct 22 18:00 test1
4 -rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 19:58 TEST.tar
5 -rw-rw-r-- 1 abdessamad abdessamad 10240 Oct 22 20:00 z
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection des entrées/sorties

Flux d'entrée/sortie standard

- Normalement **une commande** reçoit les données à partir de l'entrée standard (**STDIN**) qui est le clavier
- Elle affiche son résultat sur la sortie standard (**STDOUT**) qui est l'écran
- Il existe un troisième flux qui est la sortie standard d'erreur (**STDERR**),
⇒ Une commande lorsqu'elle génère une erreur celle-ci est affichée sur l'écran

Redirection des entrées/sorties

- La redirection des entrées/sorties permet de **rediriger la sortie d'une commande vers différents flux de données** (écran,fichier...) et aussi de **rediriger un flux de données vers l'entrée d'une commande** (clavier , fichier...)
- On peut **rediriger l'entrée standard d'une commande**
⇒ la commande va lire les données à partir d'un fichier et non à partir du clavier
- On peut aussi **rediriger la sortie d'une commande (STDOUT/STDERR) vers un fichier**
⇒ La commande va écrire son résultat ou erreur sur un fichier et ne va pas les afficher sur l'écran

Redirection de la sortie standard (STDOUT)

- Pour rediriger la **sortie d'une commande vers un fichier** on utilise le symbole **>**

commande > nom-fichier

- **Exemple1:** on a rediriger la sortie standard(STDOUT) de la commande **echo** vers le fichier **f1.txt** $\Rightarrow$ Le résultat ne sera pas afficher sur l'écran mais écrit sur le fichier **f1.txt**

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ echo Bonjour > f1.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
Bonjour
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

- Lorsqu'on utilise un seul symbole **">"** le contenu du fichier **sera écrasé**

$\Rightarrow$ **Pour éviter cela il faut utiliser un double symbole ">>"**

Redirection de la sortie standard (STDOUT)

- **Exemple2:** ici le contenu du fichier *f1.txt* sera écrasé lorsqu'on redirige la sortie standard de **echo** vers ce dernier en utilisant un seul caractère ">"

```

abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
Bonjour
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ echo les SMI3 > f1.txt
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
les SMI3

```

- **Exemple3:** Ici le contenu du fichier sera conservé et la sortie de la commande **echo** sera ajoutée à la fin du fichier grâce à l'utilisation du ">>"

```

abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
Bonjour
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ echo les SMI3 > f1.txt
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
les SMI3
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ echo , bonjour >> f1.txt
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
les SMI3
, bonjour
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$

```

Redirection de la sortie d'erreur standard (STDERR)

- La sortie d'erreur standard (STDERR) d'une commande peut être redirigé vers un fichier de la même manière qu'une sortie standard (STDOUT)
 - Le flux de données de la sortie standard est identifié par le numéro 1, alors que le flux de la sortie d'erreur est identifié par 2
 - Lorsqu'on utilise le caractère ">" pour rediriger un flux(STDOUT,STDERR) sans indiquer de numéro de flux(1,2) c'est **STDOUT** qui est redirigée par défaut
- ⇒ Pour rediriger STDERR il faut alors indiquer le numéro de flux 2

Redirection de la sortie d'erreur standard (STDERR)

- Exemple:

- 1) On commence par **exécuter une commande qui va générer une erreur**

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls  
f1.txt fich1.txt personne.txt test02.txt test1 TEST.tar z  
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f  
ls: cannot access 'f': No such file or directory  
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

- 2) Ensuite on va **rediriger la sortie de cette commande** en utilisant le caractère ">"

⇒ **On remarque que le message d'erreur est affiché sur l'écran car c'est STDOUT qui est redirigé par défaut et non STDERR**

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f > f1.txt  
ls: cannot access 'f': No such file or directory  
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt  
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection de la sortie d'erreur standard (STDERR)

- 3) Pour rediriger STDERR il faut utilisé “2>” qui signifie rediriger le flux de données de la sortie d'erreur

Ici par exemple l'erreur affichée par la commande `ls f` sera écrite(redirigée) vers le fichier *f1.txt*

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f 2> f1.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
ls: cannot access 'f': No such file or directory
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection de flux multiples

- C'est possible de rediriger STDOUT et STDERR d'une commande au même temps
- Lorsqu'on utilise le caractère ">" c'est **uniquement STDOUT** qui est redirigée et **STDERR** est affichée sur l'écran

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls
f1.txt fich1.txt personne.txt test02.txt test1 TEST.tar z
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f test1 > f1.txt
ls: cannot access 'f': No such file or directory
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
test1:
fich3.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection de flux multiples

- Lorsqu'on utilise le symbole "**2>**" c'est uniquement la sortie d'erreur qui est redirigée et **STDOUT** est affichée sur l'écran

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f test1 2> f1.txt
test1:
fich3.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
ls: cannot access 'f': No such file or directory
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

- Pour rediriger au même temps **STDOUT** et **STDERR** vers un fichier, on utilise le symbole "**&>**"

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f test1 &> f1.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
ls: cannot access 'f': No such file or directory
test1:
fich3.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```


Redirection de flux multiples

- Pour envoyer les deux flux (STDOUT et STDERR) vers deux fichiers différents on utilise les deux symboles “>” et “2>”
- L'ordre dans lequel on spécifie les deux flux n'a pas d'influence

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ ls f test1 2> error.txt > f1.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
test1:
fich3.txt
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat error.txt
ls: cannot access 'f': No such file or directory
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection de l'entrée standard (STDIN)

- La redirection de l'entrée standard (STDIN 0) **consiste à faire passer à une commande les données à partir d'un fichier et non à partir du clavier**
- Pour donner des exemples de redirection de STDIN on considère la commande **tr** qui permet de **traduire une suite de caractères en une autre suite de caractères**
- **Exemple:** Mettre en majuscule un texte qui est écrit en minuscule
⇒ on doit utiliser la commande **tr** comme suit :

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ tr 'a-z' 'A-Z'
considerant cet exemple
CONSIDERANT CET EXEMPLE
^C
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection de l'entrée standard (STDIN)

Attention:

- L'exemple précédent montre que la commande `tr` lit les données à partir du clavier
- Et si on souhaite passer à l'entrée de la commande `tr` le contenu du fichier `f1.txt` pour mettre son contenu en majuscule???
- Exemple:

```
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt
test1:
fich3.txt
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$ tr 'a-z' 'A-Z' f1.txt
tr: extra operand 'f1.txt'
Try 'tr --help' for more information.
abdessamad@abdessamad-HP-Pavillon-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

Redirection de l'entrée standard (STDIN)

- Pour forcer `tr` à lire les données à partir d'un fichier on utilise le caractère de redirection “<”

- Exemple:

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ tr 'a-z' 'A-Z' < f1.txt
TEST1:
FICH3.TXT
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```

⇒ Ici, `tr` lit les données à partir du fichier `f1.txt` au lieu de les lire à partir du clavier → on a donc **rediriger l'entrée standard de la commande `tr`**

Remarque:

- La redirection de l'entrée standard est souvent utilisée pour des commande qui n'accepte pas un nom de fichier comme argument comme le cas de `tr`

Redirection de l'entrée standard (STDIN)

Attention:

- **L'ordre des redirections compte**
⇒ **Il faut commencer par l'entrée puis la sortie**
- **Exemple:** On montre ici qu'on peut utiliser pour la même commande la redirection de STDIN et STDOUT
⇒ **tr** lit à partir du fichier *f1.txt* et écrit dans un autre fichier *error.txt*

```
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ tr 'a-z' 'A-Z' < f1.txt  
>> error.txt  
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat f1.txt  
test1:  
fich3.txt  
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$ cat error.txt  
ls: cannot access 'f': No such file or directory  
TEST1:  
FICH3.TXT  
abdessamad@abdessamad-HP-Pavilion-Laptop-15-eg0xxx:~/Desktop/SE1/test$
```